

User Interface Design

Mr. Tan Chiang Kang



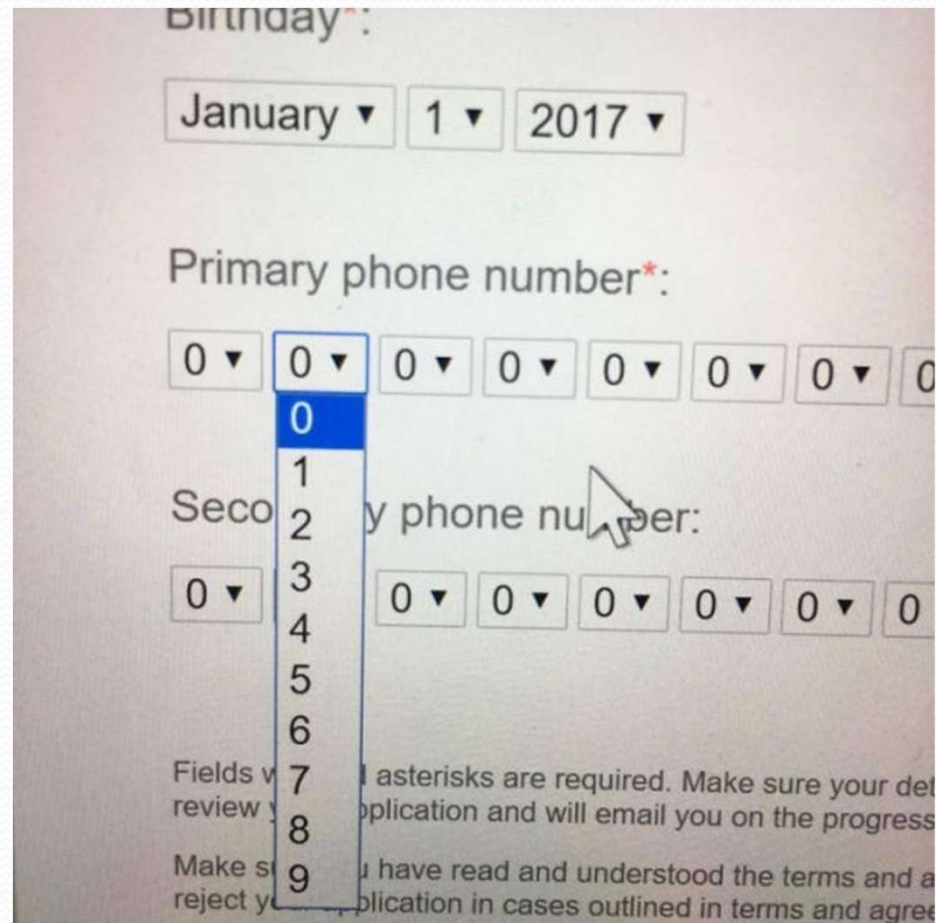


Keys to UI Design

- Designing apps is not a simple use of different elements and place them together
- User experience will affect the willingness of user downloading and installing an app
- Mobile application developers must pay attention to a few important considerations in mobile UI design
 - **Interactivity:** Good and easy interactions for the users
 - **Visual Appeal:** Design of the UI should look comfortable and appealing

Why Worry?

- A badly designed UI can be frustrating for a user
- Certain designs that were done badly can still work, but users may be forced to spend a lot more efforts to use
- Users can potentially be driven away



The image shows a portion of a web form. At the top, there is a 'Birthday' field with three dropdown menus showing 'January', '1', and '2017'. Below this is a 'Primary phone number*' field with eight dropdown menus, all showing '0'. A mouse cursor is hovering over the second dropdown menu, which has a dropdown menu open showing the numbers 0 through 9. Below the primary phone number field is a 'Secondary phone number' field with seven dropdown menus, all showing '0'. At the bottom, there is a text area with a disclaimer: 'Fields with asterisks are required. Make sure your details are correct and will email you on the progress. Make sure you have read and understood the terms and agree to our application in cases outlined in terms and agree'.

Clearly State Your Purpose

- An application must have at least a core functionality
- It can have some additional fancy features, but users should be able to tell if it is a calculator app, a messaging app, etc, even if other features are included
 - E.g. many applications includes mini games in-app
- In another words, identify core functional requirements for the application

Other Application Qualities

- Also known as non-function requirements, that describe the quality attributes of the application
- Some examples:
 - Usability – easy to learn and navigate
 - Performance – app loads quickly
 - Reliability – app available and functions with few crashes
 - Accessibility – text readable, good contrast ratio etc
 - Compatibility – app works on different devices and screen sizes
 - Security – users data should be protected

Why is interaction important?

- Mobile devices have restrictive input methods compared to traditional personal computers
- Mobile devices have smaller display area
- Mobile devices have less sophisticated input for typing and pointers
- A badly designed application can be extremely challenging for users to navigate around and to make effective use of it



5 Pillars of Interaction Design

- **Goal-driven Design:** Design based on target users' need
- **Usability:** Easy to use with minimal friction
- **Affordance & Signifiers:** Design should be familiar in terms of evident functions and their corresponding visual cues
- **Learnability:** Ease of learning to use
- **Feedback (and Response Timing):** Feedbacks are necessary to indicate interactions

Goal-driven Design

- Designs should be according to target users' behaviour and expectations
- Interviews and surveys can be conducted
- Some common tactics:
 - **Personas:** Creating fictional characters (users) and consider how they would behave
 - **User Scenarios:** Generate different user scenarios to check if the design suits the users
 - **Experience Maps:** Imagine the differences of user emotions while using the application

Usability

- An application must first be usable, before it can be good
 - If poor usability, users will look for alternatives
- Usability is typically measured by the amount of effort required to accomplish a task
- Increase usability by:
 - Removing unnecessary steps
 - Streamlining interaction flows with user
 - Using visual hierarchy, colour coding etc

Affordance & Signifiers

- Looking at this example, do you notice any problems with it?
- Why is this a problem?



Affordance & Signifiers

- **Affordance** – a function
- **Signifiers** – hints at the affordance
- These functionalities and hints should follow standard or usual practice
- User should intuitively understand what they can do or what they are expected to do with certain elements in the UI

Save

Save

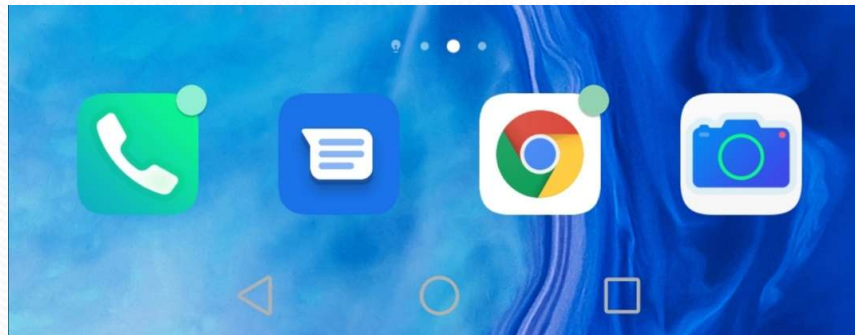
Save

Affordance & Signifiers

- Static image or video?



- Signifiers can be metaphorical



Learnability

- UI should be intuitive, such that user can quickly figure out the UI and its controls
- Users are often not interested to read through some tutorials or user manuals before using your app
- If a function is not typical then only consider using tutorials, e.g. on the first run, and place them somewhere easy to access for future reference
- Consistency and predictability are two key criteria
- Steep learning curve will put off users

Learnability

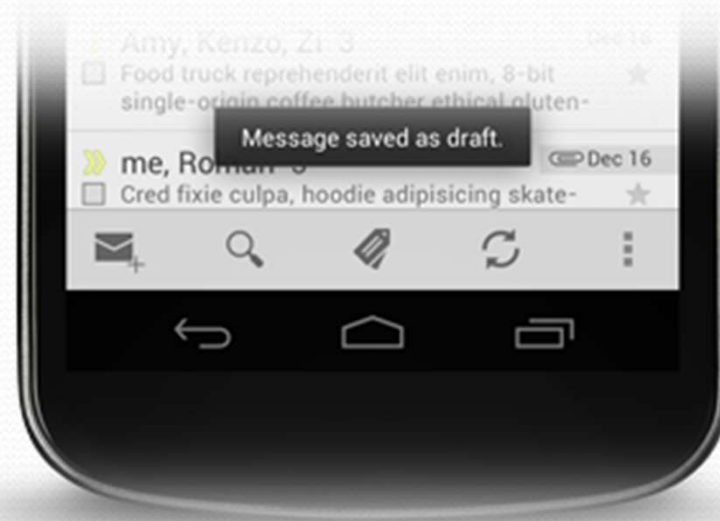
- Consistency
 - Can be achieved via consistent visuals on each page, including the behaviour of the UI
 - Example: **Similar things should look and act the same, different things should look different.**
- Predictability
 - Familiar patterns that are commonly used in similar applications, or different generations of the app itself
 - Example: **User clicks on “Settings” and expects to find the “Sign-out” option.**

Feedback (& Response Time)

- Why do we need feedback?
 - Feedback communicates the results of any interaction
- Feedback can be visual, haptic (e.g. vibration), or auditory
 - Visual feedback is most used, because it is software programmable for display.
- Feedback provide indications whether user is closer to accomplishing a task
 - If user is not, e.g. entry error, guidance can provided
- Feedback should be obvious and understandable

Feedback (& Response Time)

- This includes providing feedback when a background activity has accomplished a task



- Response of feedback should ideally be instant (expected to be well within 1 second)



5 Steps to Improve Interactions

- Roleplay the interaction
- Map out the narrative
- Simplify the steps
- Limit user choices
- Pay attention to micromoments

5 Steps to Improve Interactions

- Roleplay the interaction
 - Have 1 person showing mock interfaces to another person
 - The user should try to state what he/she wants to do
 - The interface should respond according to what an actual interface is expected to respond with



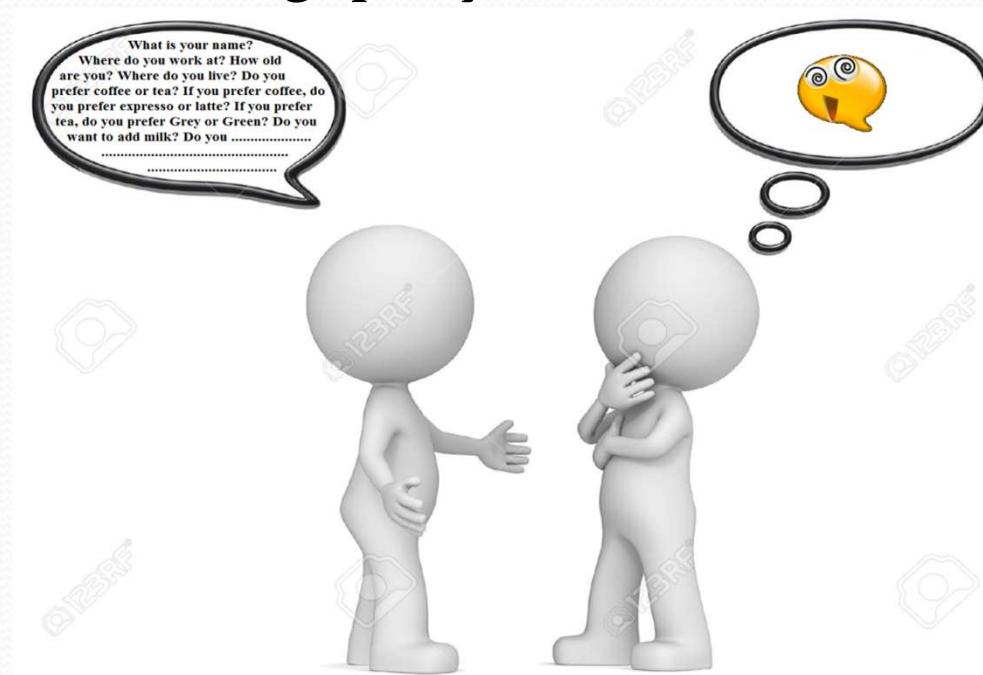


5 Steps to Improve Interactions

- Map out the narrative
 - Document the interactions between the 2 persons, including their tasks and emotions
- Simplify the steps
 - Developers often fall into the trap of trying to fit in as many functions as possible into one single UI
 - Users can easily feel overwhelmed or get confused
 - Developers should try to simplify processes whenever possible

5 Steps to Improve Interactions

- Simplify the steps
 - Example: Some complicated procedure can be disentangled into multiple logical steps rather than put together in one long query?



5 Steps to Improve Interactions

- Limit user choices
 - Users should not be given too many decisions to make in a single UI
 - Optional actions that are uncommon should usually be removed and placed elsewhere
 - Focus only on the most crucial procedure in the main UI flow
 - Specific options and overrides can always be triggered in a separate UI only when they are required

5 Steps to Improve Interactions

- Pay attention to micromoments
 - During the role play, it is possible to identify moments when the user feels unpleasant, hesitant or wishing to leave the interactions
 - Understand what users like to do, prefer to do, are willing to do or hate to do, so actions that can lead to bad user experience can be avoided
 - Example: Reduce typing, reduce scrolling, etc



Interactivity in Action

- When developing an app, some guidelines on good interactivity design will help provide better user experience
- The following areas should be taken care of:
 - Input
 - Navigation
 - Data & Content Management

Input

- Due to the smaller device sizes, mobile devices lack sophisticated keyboards and pointing devices in general
- Most mobile devices rely on screen keypad for typing
- Stylus is not always available
- Requests for users to provide input as much as possible



Input – Registration

- Signing up is a painful process on mobile devices
- Users are often required to enter a lot of details during registration
- The registration process should be simplified and only request for bare necessity information (focus on user ID, password, email for example)
- Existing social media accounts from users can be used for signing up instead (Google, Facebook, etc)
- A free trial can be offered for users to experience the app before sign-up

Input – Minimise Typing

- Typing is very troublesome and time consuming on mobile devices, and it is prone to typing errors
- Whenever possible, efforts should be made to reduce typing:
 - Only ask the users to type unpredictable data inputs
 - Provide a relevant keyboard for the users (e.g. numerical)
 - Provide default values and auto-complete feature

Input – Buttons

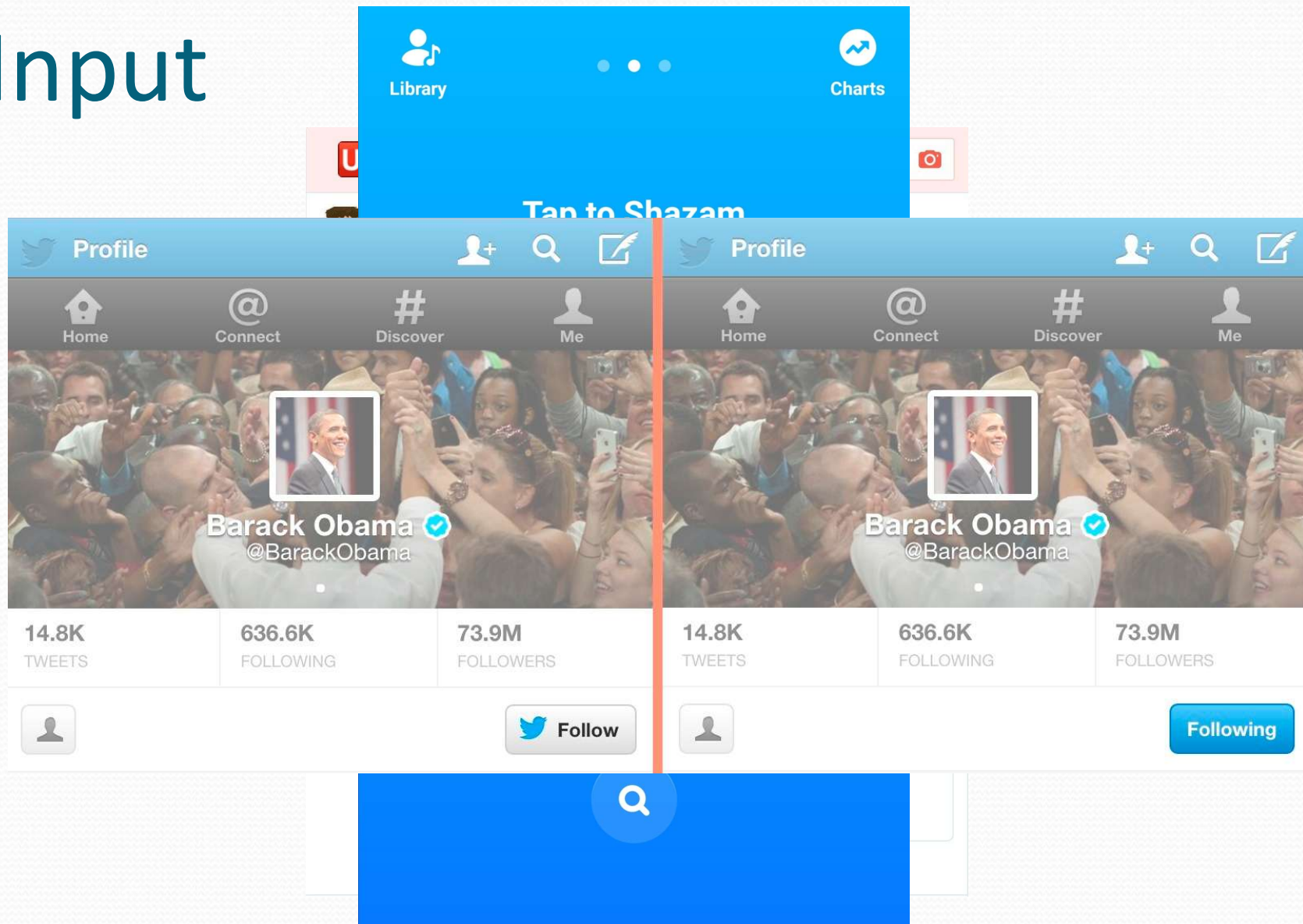
- Buttons are a common input for users to confirm or make selection
- The usual way to trigger the buttons is via a finger touch, which easily spans 10-15 mm wide
- Design of buttons need to be large enough to cover this touch area
- Ensure padding space between buttons are spacious enough so users will not easily click on the wrong button
- You can even provide a huge button!



Input

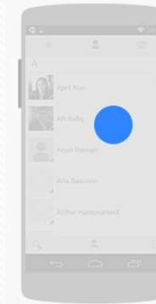
- Other common tactics:
 - Convenience / UI friendliness:
 - Swipe for some actions
 - Use sliders to help change a selection
 - Allow actions to be undo-able
 - Reduce UI clutter / improve UI look & feel
 - Use morphing controls on some true/false actions
 - Use discoverable controls
 - Use expandable inputs
 - Reduce input
 - Provide feedback on progress visually
 - Provide notifications of updates visually

Input



Input - Gestures

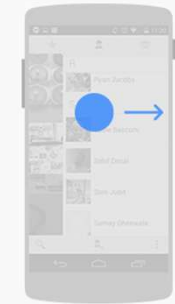
- Gestures are a series of important input method for modern day touchscreen-based mobile devices
- They allow certain actions to be more easily achieved
- Other than standard gestures, developers can also define specific swipe patterns to create new gestures



Touch



Long press



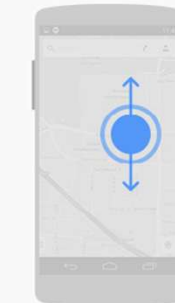
Swipe or drag



Long press drag



Double touch



Double touch drag



Pinch open



Pinch close



Navigation

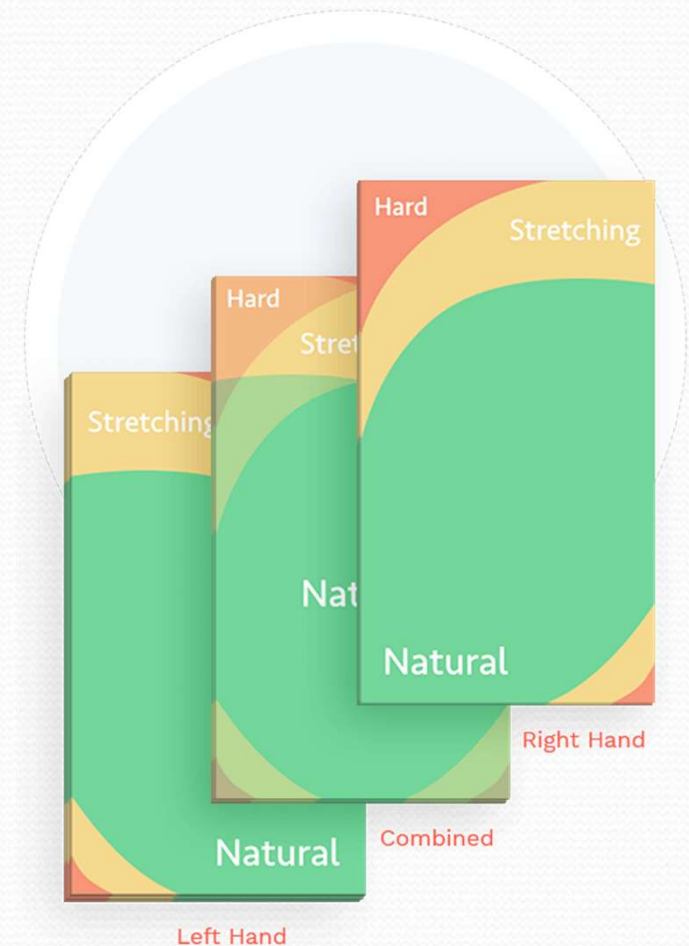
- Without more area to display contents, users often need to navigate around to get what they need
- It is very important to ensure that navigation is not a hassle for the users and navigation actions to all processes are reduced to as few as possible
- Navigation should be easy, fast, intuitive and not confusing

Navigation – Going Back

- Many apps break an action into many small pages, allowing users to go through them sequentially
- It should be easy for them to go back or undo an action
- iOS apps usually offer a back button on the top left-hand corner of the screen
- Android devices usually come with a back button on the device that users can directly use

Navigation – Thumb Friendly

- Users often use their mobile device with a single hand
- There are different levels of reach within the screen
- Place the most crucial components in the most natural zones for easy access
- Avoid placing icons and buttons at zones that are the most difficult or awkward to touch



Navigation – In-app Search

- In-app searches can help users navigate to targeted contents more swiftly
 - The search box must be obvious and easy to find
 - Use auto-complete to help users key in search terms more quickly
 - Use high quality search indexing to get relevant results
 - Avoid no match found pages, and suggest closest results as much as possible



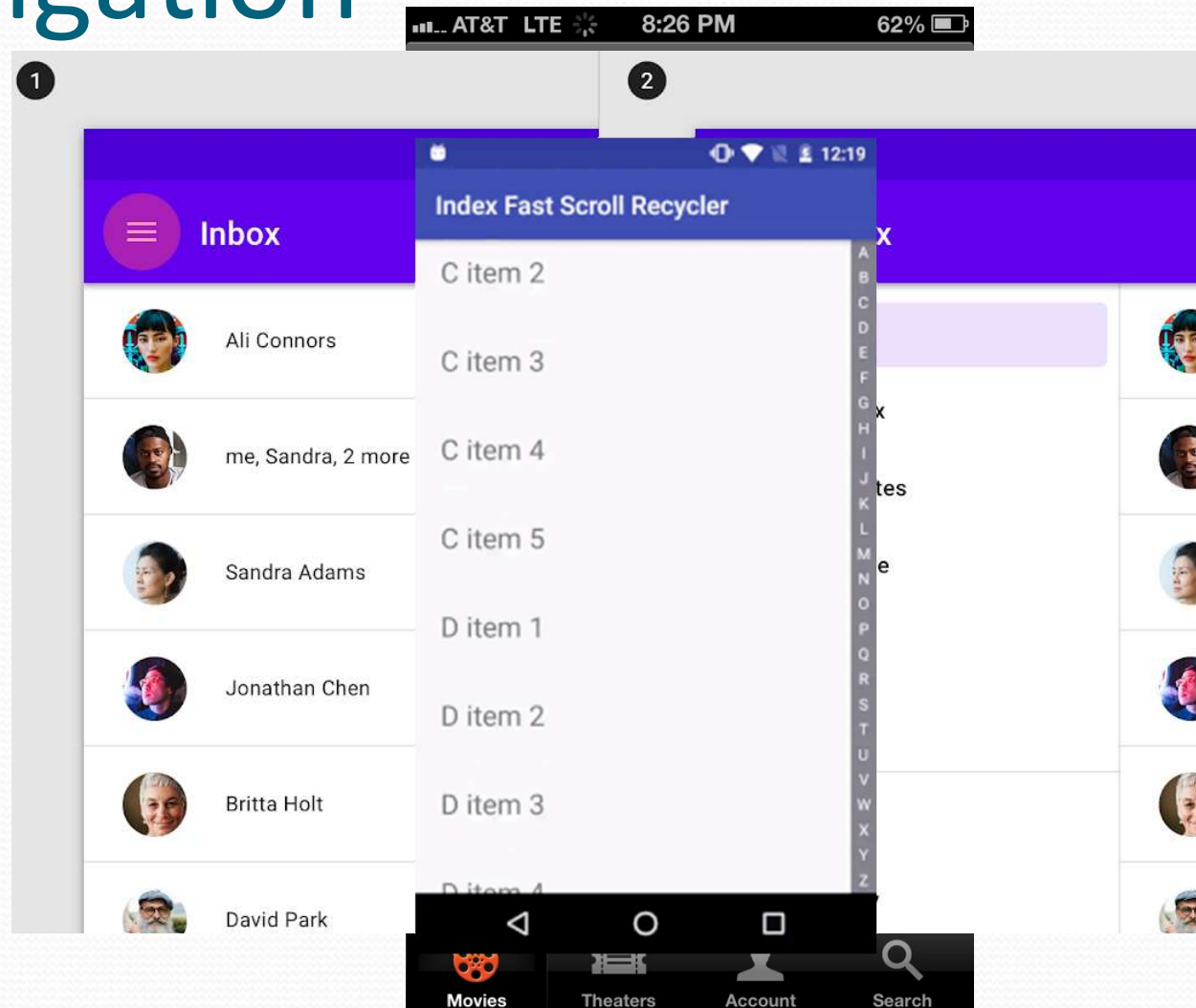
Navigation – Consistency

- Applications usually use one single theme throughout
- Consistent colour scheme allows users to not feel confused or disoriented
 - Erratic changes of layout and colour scheme can easily cause users to feel dizzy and uncomfortable
- If the application exists also on other platforms, including web application, try to stick to the same colour scheme

Navigation

- Some other common tactics:
 - Use a walkthrough tutorial or coach marks for new users
 - Use overflow menus (e.g. options menu, drawer)
 - Use fixed navigation that will stay throughout
 - Offer search bar/option
 - Use advanced scrollbars with fast access indexes
 - Implement side swipes to swap between contents
 - Use popover dialog boxes to display some additional information or guide user

Navigation



Data & Content Management

- Data and contents, if displayed in a better way, can provide better experiences to the users
- Some tips that can be used:
 - Use full-screen modes for certain suitable actions (e.g. games, viewing videos, etc)
 - Use circles on avatar thumbnails to differentiate it from medias (e.g. photos)
 - Place some instructions or descriptions on pages that were empty, rather than really leaving them blank!
 - Use RecyclerView to efficiently display a large sets of data dynamically in a pre-defined way

Visual Appeal

- Humans are generally attracted to beautiful things
- Given a choice, people select pleasing products over ugly ones



Visual Appeal

- A developer may not be a graphic designer to produce the most beautiful products
- However, a developer should be able to at least produce average UI that are not ugly
- Some key considerations in UI visual design are:
 - **Layout:** Selecting the right number of elements to be placed in the UI
 - **Colours:** Selecting the right colour combinations

Layout

- Usually, the developer selects the most crucial information to be fitted into a single page
- Avoid including too much information into a single page - you can utilise other pages as well
- Try to display one whole page without truncation if possible

My Simple App

Ask for something:

Show something:

Something	Something
Something	Something
Something	Something
Something	Something
Something	Something
Something	Something
Something	Something
Something	Something

Cancel OK

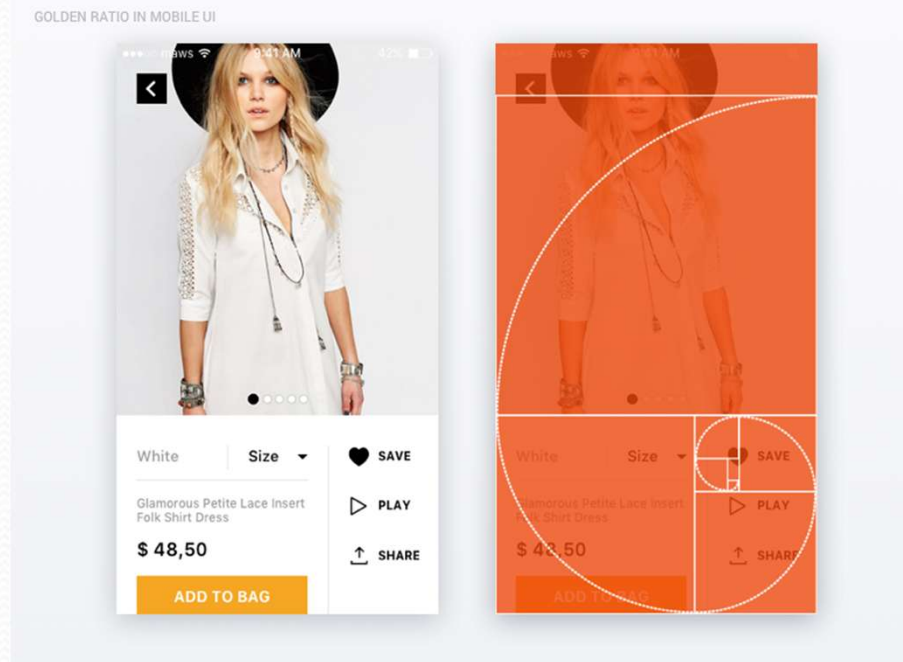


Layout

- When many functions are available and it is impossible to fit everything into a single page, multiple pages should be used
- It is best the separated pages follow a similar basic layout design
- Sufficient blank spaces should be included in the layout, without giving a feeling that the components are cramped together

Golden Ratio

- A rough idea of golden ratio is 1:1.618
- Using this idea, the developer can spend effort to place important functions and make use of available space accordingly





Use More Images than Texts

- Use more graphics and icons than using texts whenever possible
- Choose fonts that are very clear and easy to read over fancy styles
- The images should be at high enough resolution to show clarity
- Image slicing can be used to load higher resolution images, to keep users engaged
- Make sure that the images are in right aspect ratio



Colours

- Colours selection is a small action with a huge effect – select the wrong colours and it can lead to very bad outcome
- For reading purposes, colours are only appealing to look at if they have sufficient contrast, hence designers generally only use high contrast colour combinations, e.g. bright colours and dark colours
- Complicated patterns may look nice when presented as an image, but using one as a background may not be always appealing, why?

Colours

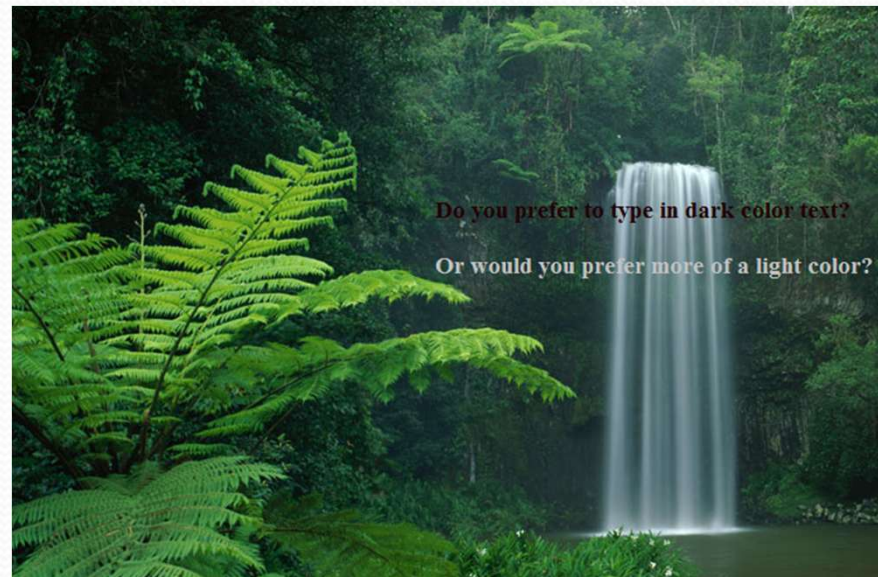
- Which one do you prefer? Why?

Do you like this?

How about this?

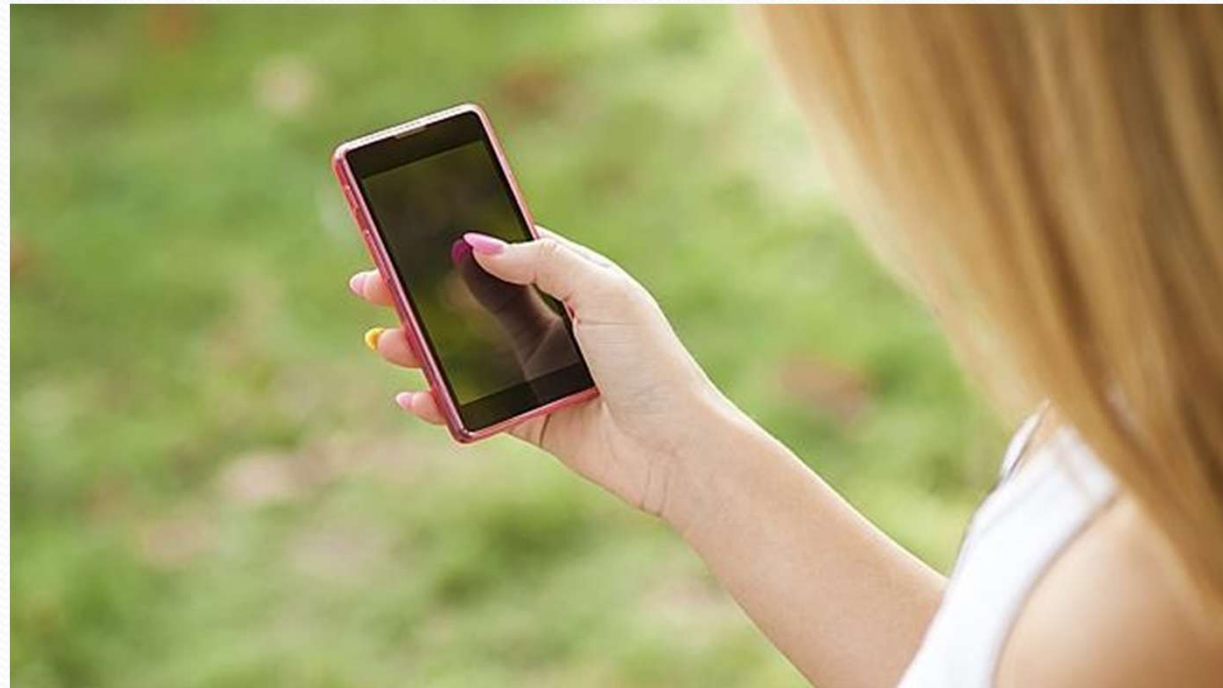
And this?

- And with a complicated background, what problem can it present?



Colours

- Colour combinations with poor contrast is difficult to be view indoor and almost impossible to read when in outdoor environments



Contrast ratio

- Refers to the difference in brightness between foreground and background elements (or most commonly between text and its background)
- The World Wide Web Consortium (W3C) recommends minimum contrast ratios in its Web Content Accessibility Guidelines (WCAG):
 - 4.5:1 for normal text
 - 3:1 for large text
- Good contrast improves usability/readability and reduces eye strain
 - Friendlier to users with visual impairments, e.g. colour blindness or low vision.



Follow Latest Design Trends

- Do also look at the current layout and colour theme used by other applications
- Following what others do is sometimes a good and safe bet, and your design can remain current
- It can more easily meet the expectation of users as they try to get themselves familiarised with your app



Simplicity

- Simple is not necessarily a bad thing
- In mobile applications, simplicity often provides the best experience to users
- Adding too many fancy elements sometimes do not help in making the application more attractive or more usable
- In fact, they may bring about opposite effect!

Resolution Independence

- Our smartphones come in various sizes with different designs (especially Android)
- This is exciting for the market but often a hazard for application developers
- For UI, the screen size is crucial, but it can vary from device to device, including its resolution
- Some of the examples we learned in the Practical Lab Sessions used hard-coded pixel values, do you think this is a good practice?

Resolution Independence

- Whenever possible, hard-coded pixel values should be avoided in UI design
- One may use relative parameters such as those related to layout size (**wrap_content**) or following their parent size (**match_parent**) where appropriate
- We can also implement layout that allows alignment relative to the edge/border of the screen
- Obtaining the width and height of the screen is useful for relative placement of items or sizing an item

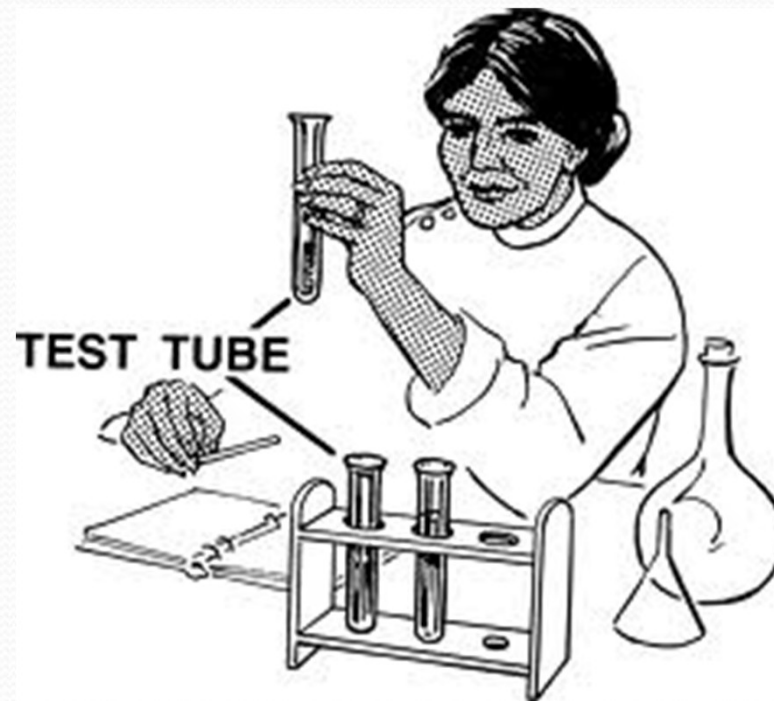


Resolution Independence

- Today, modern mobile devices range from 4 inches to 10 inches
- The design should aim to be **viewable** and **usable** on all different screen sizes
 - Users should not have difficulty reading the contents on the screen even if they have a small screen
 - Users should not have difficulty using it, such as inserting an input, selecting an option on the screen etc

How to ensure it works well?

- Due to screen size differences, it is recommended to test your UI on emulators with different screen sizes



What to remember?

- An application is developed to be used by the users
- Always remember to think on behalf of the users
- Pay attention to users' preferences and need (**must try hard to suppress the developer's preference or bias!**)
- If you are uncertain, ask the users, or let them try it out

What's Next?

- Animations